SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT TEMPLATE

1. Student Details

Student Name	Pranav Chopade
PRN	202501110025
Division	A
Batch	A1
Roll No.	202501110025
Date of Submission	01-11-2026

2. Practical Details

Practical No.	7
Practical Title	WRITE A PROGRAM ON POLYMORPHISM
Date Performed	01-11-2026
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

Write a C++ program to demonstrate runtime polymorphism using function overriding and a virtual function. Create a base class Shape and derived classes Rectangle and Circle. Use a base-class pointer to call the overridden area() function for each derived-class object.

4. Objectives

1. Understand the concept of polymorphism in C++. 2. Implement function overriding in derived classes. 3. Use a virtual function in the base class. 4. Demonstrate runtime polymorphism using a base-class pointer.

5. Expected Learning Outcomes

After completing this practical, the student will be able to: 1. Explain polymorphism. 2. Differentiate compile-time and runtime polymorphism. 3. Use virtual functions and function overriding. 4. Apply base-class pointers to achieve dynamic method dispatch.

6. Theory / Concept

Polymorphism: An OOP feature that allows one interface or function name to represent different behaviours.

Compile-time Polymorphism: Resolved by the compiler, commonly using function overloading and operator overloading.

Runtime Polymorphism: Resolved during program execution using virtual functions and function overriding.

Virtual Function: A base-class member function declared with the virtual keyword so an overridden version can be selected at runtime through a base pointer or reference.

Function Overriding: A derived class provides its own implementation of a base-class virtual function.

7. Algorithm / Pseudocode

START
1. Define a base class Shape with a virtual area() function.
2. Define Rectangle derived from Shape and override area() to calculate length × breadth.
3. Define Circle derived from Shape and override area() to calculate 3.14 × radius × radius.
4. Create one Rectangle object and one Circle object.
5. Declare a Shape pointer.
6. Assign the pointer to the Rectangle object and call area().
7. Assign the pointer to the Circle object and call area().
8. Display the calculated areas.
STOP

8. Technology / Tools / Libraries Used

Programming Language	C++
IDE / Platform	Dev C++
Libraries / Packages	iostream

9. Implementation / GitHub Repository

GitHub Repository URL	https://github.com/pranav-4797/OOP-Practical-Assignment
Repository Name	OOP-Practical-Assignment
Practical Folder	Practical 7
Branch Used	main
Commit ID / Latest Commit	Not provided

Repository Contents Used

The source file is provided separately as “Practical_7_Polymorphism.cpp”. Add it to the Practical 7 folder in the repository if required.

10. Test Cases / Input and Output

Test	Object	Expected Output	Status
1	Rectangle(10, 5)	Area of Rectangle: 50	Pass (sample)
2	Circle(7)	Area of Circle: 153.86	Pass (sample)

11. Results / Observations

The program demonstrates runtime polymorphism. The same base-class pointer calls the overridden area() function of Rectangle and Circle objects, producing different area calculations depending on the object assigned to the pointer.

12. Source Code

```
#include <iostream>
using namespace std;

class Shape {
public:
virtual void area() {
cout << "Area of a general shape" << endl;
}
};

class Rectangle : public Shape {
private:
int length, breadth;

public:
Rectangle(int l, int b) {
length = l;
breadth = b;
}

void area() {
cout << "Area of Rectangle: " << length * breadth << endl;
}
};

class Circle : public Shape {
private:
float radius;
```

```
public:
Circle(float r) {
radius = r;
}

void area() {
cout << "Area of Circle: " << 3.14 * radius * radius << endl;
}
};

int main() {
Shape *ptr;

Rectangle r(10, 5);
Circle c(7);

cout << "RUNTIME POLYMORPHISM" << endl;

ptr = &r;
ptr->area();

ptr = &c;
ptr->area();

return 0;
}
```

Screenshots / Output

Representative terminal-style output. Run the program in Dev-C++ to capture an actual IDE screenshot if required.

13. Conclusion

This practical demonstrates runtime polymorphism in C++ using virtual functions, function overriding, and a base-class pointer. The area() function produces different results for Rectangle and Circle objects through the same interface.

14. Assignment Questions

Q1. What is polymorphism?
Answer: Polymorphism means one interface can provide different behaviours depending on the object or context.

Q2. What are the two main types of polymorphism?
Answer: Compile-time polymorphism and runtime polymorphism.

Q3. What is a virtual function?
Answer: A base-class function declared with virtual so the overridden derived-class function can be selected at runtime.

Q4. What is function overriding?
Answer: When a derived class provides its own implementation of a function declared in its base class.

Q5. Why is a base-class pointer used?
Answer: It can point to derived-class objects and invoke their overridden virtual functions.

15. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	Pranav Chopade
PRN	202501110025
Signature	